CAPITOLO 3

Modelli formali per il calcolo

1. *Funzioni Ricorsive*

Anche per il modello formale di calcolo definito dalle funzioni ricorsive (parziali o totali che siano) bisogna codificare un algoritmo per risolvere un dato problema nel formalismo dato. Innanzitutto, dobbiamo cercare di costruire un algoritmo che risolva il nostro problema in modo ricorsivo. Questo significa che il risultato deve essere ottenuto mediante composizione (composizione di funzioni, ricorsione primitiva, minimizzazione o μ-ricorsione) ed utilizzando solamente la funzione stessa che stiamo definendo ed eventualmente altre funzioni ricorsive che siano note o a loro volta definibili in modo ricorsivo. Per semplicità ricordiamo alcune funzioni già date nel libro.

La funzione che fornisce il resto è *mod(x,y)* mentre un'altra funzione utile è $\overline{\mathtt{sg}}(z)$; questa infatti, applicata a *mod(x,y)* mi restituisce 1 se il numero y è divisore di x, 0 altrimenti. Ricordiamo che:

Somma:
$$\begin{cases} +(x,0) = x \\ +(x,y+1) = S(+(x,y)) \end{cases}$$

Moltiplicazione:
$$\begin{cases} \cdot(x,0) = 0 \\ \cdot(x,y+1) = +(\cdot(x,y),x) \end{cases}$$

Sottrazione: $-(x,y)$: Resituisce $x-y$ se $x > y$, restituisce 0 altrimenti
$$\begin{cases} -(x,0) = x \\ -(x,x) = 0 \\ -(x,y+1) = D(-(x,y),1) \end{cases}$$
where $D(0) = 0$ and $D(y+1) = y$.

Segno:
$$\begin{cases} \mathtt{sg}(0) = 0 \\ \mathtt{sg}(y+1) = S(0) \end{cases}$$
Inoltre definiamo $\overline{\mathtt{sg}}(x) = -(1, \mathtt{sg}(x))$.

Modulo:
Assumiamo che $\mathtt{mod}\,(x,0) = 0$.
$$\begin{cases} \mathtt{mod}\,(0,x) = 0 \\ \mathtt{mod}\,(y+1,x) = S(\,\mathtt{mod}\,(y,x)) \cdot \mathtt{sg}(-(x, S(\,\mathtt{mod}\,(y,x)))) \end{cases}$$

Divisione intera:
Assumiamo che $\mathtt{div}(x,0) = 0$.
$$\begin{cases} \mathtt{div}(0,x) = 0 \\ \mathtt{div}(y+1,x) = \mathtt{div}(y,x) + \overline{\mathtt{sg}}(-(x, S(\,\mathtt{mod}\,(y,x)))) \end{cases}$$

Sommatoria:
Sia f una funzione ricorsiva primitiva binaria, allora definiamo la funzione $\Sigma_{z<y} f(x,z) = \Sigma_{z=0}^{y-1} f(x,z)$:
$$\begin{cases} \Sigma_{z<0} f(x,z) = 0 \\ \Sigma_{z<y+1} f(x,z) = +(f(x,z), \Sigma_{z<y} f(x,z)) \end{cases}$$

Produttoria:
Sia f una funzione ricorsiva primitiva binaria, allora definiamo la funzione $\Pi_{z<y} f(x,z) = \Pi_{z=0}^{y-1} f(x,z)$:
$$\begin{cases} \Pi_{z<0} f(x,z) = S(0) \\ \Pi_{z<y+1} f(x,z) = \cdot(f(x,z), \Pi_{z<y} f(x,z)) \end{cases}$$

1.1. Esercizi svolti.

ESERCIZIO 1.1. *Si definisca la funzione primitiva ricorsiva* $f : \mathbb{N} \to \mathbb{N}$ *che conta il numero di occorrenze della cifra* 5 *in un numero naturale dato in base decimale.*

SOLUZIONE. La risoluzione di tale esercizio è alquanto più delicata rispetto agli esercizi fin qui risolti. In questo caso dobbiamo trovare un modo per analizzare una alla volta le cifre di un numero decimale. A questo scopo ci vengono in aiuto le funzioni `mod` e `div`, infatti eseguendo la divisione intera tra un numero decimale e 10 si ottiene lo stesso numero privato della cifra meno significativa, mentre calcolando il resto della stessa divisione si ottiene la proprio la cifra più a destra del numero (si noti che tale fatto vale per numeri in qualsiasi base purché vengano divisi per la loro base). Dunque possiamo risolvere l'esercizio costruendo una funzione che ricorsivamente scinda il numero nelle sue cifre e controlli quante di queste siano 5. Diamo quindi la definizione formale

$$\begin{cases} f(0) = f(1) = f(2) = f(3) = f(4) = f(6) = f(7) = f(8) = f(9) = 0 \\ f(5) = 1 \\ f(x) = +(f(\texttt{div}(x,10)), f(\texttt{ mod }(x,10))) \end{cases}$$

□

ESERCIZIO 1.2. *Si definisca una funzione primitiva ricorsiva* $f : \mathbb{N} \to \mathbb{N}$ *tale che se* $n \in \mathbb{N}$ *è l'input, essa restituisca la cifra decimale più a sinistra del numero* n.

SOLUZIONE. Si noti che l'esercizio richiede la cifra più a sinistra mentre noi sappiamo già selezionare quella più a destra. Volendo sfruttare questa capacità possiamo notare che il risultato della nostra funzione applicato a n è lo stesso di quello della funzione applicata al numero ottenuto da n togliendo la cifra più a destra, poiché tale operazione non modifica la cifra più a sinistra. Così la definizione formale della

funzione è

$$\begin{cases} f(0) = 0 \\ f(1) = 1 \\ f(2) = 2 \\ f(3) = 3 \\ f(4) = 4 \\ f(5) = 5 \\ f(6) = 6 \\ f(7) = 7 \\ f(8) = 8 \\ f(9) = 9 \\ f(x) = f(\texttt{div}(x, 10)) \end{cases}$$

Abbiamo inserito anche il caso $f(0)$ per considerare la situazione in cui l'input iniziale sia 0. □

ESERCIZIO 1.3. *Si definisca la funzione primitiva ricorsiva* $f : \mathbb{N} \to \mathbb{N}$ *che dato un numero decimale in input restituisce il reverse dello stesso, ad esempio se* $x = 247$ *allora la funzione deve restituire* 742.

SOLUZIONE. In questo caso, per risolvere l'esercizio, dobbiamo cercare di sfruttare il significato della posizione delle singole cifre nel numero. Cioè un numero decimale è ottenuto come somma di ogni cifra componente moltiplicata per 10 elevato al valore indicante la posizione della cifra nel numero, meno un'unità, contando da destra; nell'esempio si ha $247 = 2 \cdot 10^2 + 4 \cdot 10^1 + 7 \cdot 10^0$. A questo punto potendo scindere il numero nelle sue cifre componenti possiamo costruire il numero reverse dell'input trovando i giusti esponenti di 10 per ogni cifra. Ad esempio per ottenere 742 la cifra 7 va moltiplicata per 10^2 dove 2 è la lunghezza del numero 247 meno un'unità, potremmo poi completare l'operazione sommando il risultato a ciò che si ottiene eseguendo ricorsivamente gli stessi passi sul numero 24. Si noti che per fare ciò che abbiamo appena descritto abbiamo bisogno di una funzione che calcoli la lunghezza del numero e di una che esegue l'effettivo reverse. Definiamo formalmente le due funzioni

$$\begin{cases} f(0) = 0 \\ f(x) = +(f(\texttt{div}(x, 10)), \cdot(\texttt{mod}(x, 10), 10^{(\texttt{lung}(x)-1)})) \end{cases}$$

$$\begin{cases} \texttt{lung}(0) = 0 \\ \texttt{lung}(x) = S(\texttt{lung}(\texttt{div}(x, 10))) \end{cases}$$

□

ESERCIZIO 1.4. *Si definisca la funzione primitiva ricorsiva* $\text{ndiv} : \mathbb{N} \to \mathbb{N}$ *che, dato* $x \in \mathbb{N}$*, fornisce il numero dei suoi divisori.*

SOLUZIONE. Per risolvere il problema utilizziamo delle proprietà dei divisori di un numero intero. Sappiamo che un numero è divisore di x se la divisione tra x e tale numero fornisce resto 0. Poiché possediamo già una funzione ricorsiva che fornisce il resto di una divisione intera possiamo pensare di contare tutti i numeri tali che la divisione di x con essi dia 0.

Vediamo allora la formalizzazione dell'algoritmo come funzione ricorsiva definita per casi.

$$\begin{cases} \mathtt{ndiv}(0) = 0 \\ \mathtt{ndiv}(x) = (\Sigma_{y<x+1}\ \overline{\mathtt{sg}}(\ \mathtt{mod}\ (x, y))) - 1 \end{cases}$$

Si noti che la sottrazione di un'unità è necessaria perché, per come è costruita la funzione, `ndiv` conterebbe come divisore anche lo zero che, come sappiamo, non è divisore di nessun naturale. □

ESERCIZIO 1.5. *Si definisca la funzione ricorsiva primitiva* $\mathrm{primo} : \mathbb{N} \to \mathbb{N}$ *che, dato* $x \in \mathbb{N}$*, restituice* 1 *se* x *è primo,* 0 *altrimenti.*

SOLUZIONE. Per risolvere il problema dobbiamo identificare una caratteristica specifica dei numeri primi. Essa può essere la definizione stessa di numero primo, cioè un numero è primo se ha come unici divisori 1 e se stesso, questo significa che il numero di divisori è esattamente due. Infatti un numero è primo se e solo se ha esattamente due divisori. Possiamo allora utilizzare la funzione definita nell'esercizio 1.4 contando il numero di divisori di x dato in input e restituendo 1 se tale numero è proprio 2.

Dunque la definizione formale per casi è

$$\begin{cases} \mathtt{primo}(0) = 0 \quad \mathtt{primo}(1) = 1 \\ \mathtt{primo}(x) = \overline{\mathtt{sg}}(-(\mathtt{ndiv}(x), 2)) \end{cases}$$

Cioè determiniamo il numero di divisori con `ndiv`, a questo togliamo 2; se il risultato è zero il numero è primo e restituiamo 1, altrimenti restituiamo 0. □

ESERCIZIO 1.6. *Si definisca la funzione primitiva ricorsiva* $p : \mathbb{N} \to \mathbb{N}$ *che dato* $x \in \mathbb{N}$ *restituisce l'x-esimo numero primo (si assuma* $p(0) = 0$*).*

SOLUZIONE. Per risolvere tale problema dobbiamo trovare un modo per contare i numeri primi fino ad arrivare all'x-esimo. Per fare questo potremmo partire da 0 e per ogni numero naturale controllare se è primo. In caso affermativo togliamo un'unità al numero x altrimenti lo lasciamo invariato. Terminando tale procedimento quando x diventa nullo individuiamo il valore cercato. Vediamo la definizione formale e poi ne descriviamo l'esatto funzionamento. Definiamo

$$p(x) = q(x, 0)$$

dove $q : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ è tale che

$$\begin{cases} q(x, y) = q(-(x, \mathtt{primo}(S(y))), S(y)) \\ q(0, y) = y \end{cases}$$

L'introduzione della nuova funzione con due parametri serve per gestire contemporaneamente sia il contatore dei numeri primi x sia il valore che alla fine sarà il numero cercato y. Si noti comunque che tale tipo di ricorsione è insolito perché l'utilizzo della funzione ricorsiva di base successore fa in modo che uno dei parametri aumenti continuamente. La terminazione è comunque garantita dal fatto che, vista l'infinitezza dell'insieme dei numeri primi, per ogni x il procedimento arriva

alla conclusione perché sempre riusciremo a trovare almeno x numeri primi. □

ESERCIZIO 1.7. *Si definisca la funzione ricorsiva primitiva* $\text{expf} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ *che, dato* $x, y \in \mathbb{N}$*, restituisce l'esponente del y-esimo numero primo nella fattorizzazione in numeri primi di* x*.*

SOLUZIONE. In questo caso dobbiamo cercare di contare il numero di volte in cui il numero $p(y)$, ottenuto mediante la funzione definita nell'esercizio 1.8, è contenuto nel numero x. Definiamo formalmente la funzione

$$\begin{cases} \texttt{expf}(0, y) = 0 \\ \texttt{expf}(x, y) = +(\overline{\texttt{sg}}(\texttt{mod}\,(x, p(y))), \texttt{expf}(\texttt{div}(x, p(y)), y)) \end{cases}$$

Cioè se $p(y)$ è divisore di x aggiungiamo 1 al risultato della funzione richiamata sul quoziente tra x e $p(y)$, e y. In caso contrario sommiamo una sequenza di zeri. □

ESERCIZIO 1.8. *Si definisca la funzione primitiva ricorsiva* $p : \mathbb{N} \to \mathbb{N}$ *che dato* $x \in \mathbb{N}$ *restituisce l'x-esimo numero primo usando la minimizzazione (si assuma* $p(0) = 0$*).*

SOLUZIONE. Si noti come la funzione richiesta sia anche definibile per minimizzazione o μ-ricorsione nel seguente modo:

$$\begin{array}{lcl} p(0) & = & 0 \\ p(1) & = & 2 \\ p(n+1) & = & f(p(n)) \end{array}$$

dove: $f(x) = \mu z.\ \texttt{and}\,(\texttt{sg}(-(z, x)), \texttt{primo}(z))$ e $\texttt{and}\,(x, y) = 1$ se $x = y = 1$, $\texttt{and}\,(x, y) = 0$ altrimenti. □

ESERCIZIO 1.9. *Definire la funzione primitiva ricorsiva* $\text{ifcubo} : \mathbb{N} \to \{0, 1\}$ *che restituisce* 1 *se il valore* x *dato in input è un cubo perfetto,* 0 *altrimenti.*

SOLUZIONE. Per dimostrare che x è un cubo perfetto dobbiamo trovare un valore, il quale sarà sicuramente minore o uguale a x stesso, tale che moltiplicato per sè stesso tre volte dia esattamente x. Per prima cosa definiamo la seguente funzione ausiliaria:

$$\texttt{diff}(x, y) = +(-(x, y), -(y, x))$$

che restituisce la differenza tra i valori x e y, infatti se $x = y$ entrambe le operazioni $-$ restituiscono 0 (e la differenza è effivamente 0, altrimenti abbiamo $x > y$ (e in tal caso $-(x, y)$ è la differenza, mentre $-(y, x)$ è 0) o $y > x$ (e in tal caso $-(y, x)$ è la differenza, mentre $-(x, y)$ è 0). Per trovare tale valore possiamo utilizzare la funzione di minimizzazione,

$$\begin{cases} \texttt{ifcubo}(0) = \texttt{ifcubo}(1) = 1 \\ \texttt{ifcubo}(x) = \texttt{sg}(\mu y.\texttt{diff}(x, \cdot\,(y, \cdot\,(y, y))) = 0) \end{cases}$$

Quindi se esiste un y il cui triplo prodotto è x l'operatore di minimalizzazione lo restituisce e la funzione segno viene applicata a un valore diverso da zero dando 1. In caso contrario, la minimizzazione diverge nella ricerca.
In realtà, essendo la funzione cubo crescente, se un valore y esiste, allora questo è minore di x. questo significa che possiamo in realtà controllare solo per ogni valore minore di x se il suo triplo prodotto meno x è esattamente zero. Se entro x non troviamo tale valore allora non esiste sicuramente e alla fine della computazione possiamo restituire zero.

$$\texttt{ifcubo}(x) = \texttt{sg}(-(x, (\mu y.\, \texttt{and}\, (\texttt{sg}(\texttt{diff}(\cdot\,(y, \cdot\,(y, y)), x)), \texttt{sg}(-(x, y))) = 0)))$$

ovvero, finché $x \neq y^3$ e $x > y$ entrambi i valori della funzione and restituiscono 1, la minimizzazione finisce quando la funzione `diff` restituisce 0 (ovvero abbiamo trovato y tale che $y^3 = x$, e in tal caso $y < x$ e quindi il risultato finale è 1), oppure quando $-(x, y) = 0$, ovvero $y \geq x$, ed in tal caso allora anche il segno finale è 0. □

Esercizio 1.10. *Si definisca la funzione primitiva ricorsiva* ifscubi : $\mathbb{N} \to \{0, 1\}$ *che restituisce* 1 *se il valore* x *dato in input è somma di cubi,* 0 *altrimenti.*

Soluzione. La risoluzione di tale esercizio risulta semplificata se si utilizza la funzione definita nell'esercizio 1.9 Infatti per ogni valore y minore del'input x controlliamo se sia y che $(x - y)$ sono cubi mediante la funzione `ifcubo`. Definiamo dunque formalmente la funzione

$$\begin{cases} \texttt{ifscubi}(0) = 1 \\ \texttt{ifscubi}(x) = \texttt{sg}(\mu y.\, \overline{\texttt{sg}}(\cdot\,(\texttt{ifcubo}(y), \texttt{ifcubo}(-(x, y))))) = 0) \end{cases}$$

Come nell'esercizio precedente possiamo rendere terminante la ricerca, fermandola quando superiamo x:

$$\texttt{ifscubi}(x) = \texttt{sg}(-(x, (\mu y.\, \texttt{and}\, (\overline{\texttt{sg}}(\cdot\,(\texttt{ifcubo}(y), \texttt{ifcubo}(-(x, y)))))), \texttt{sg}(-(x, y)) = 0)))$$

□